

WashO

Code Guide for the Viva

Which file to open when the teacher asks, what it does (explained so a 10-year-old understands), and a real example for each.

How to open a file at a line: open the WashO folder in VS Code, press **Ctrl + P** and type the file name, then press **Ctrl + G** and type the line number.

Part 1: Files to open · Part 2: Simple explanations · Part 3: Examples

Part 1: Important files to open

Keep this page open during the viva. Each row gives the file and the line to jump to.

#	Open this file	Line	What it is
1	washo/settings.py	70	Settings: how WashO connects to the database
2	washo/urls.py	13	URL map: which address opens which page
3	accounts/models.py	37	Custom user: login with a mobile number
4	accounts/permissions.py	13	Role check: who is allowed into which page
5	catalog/models.py	109	Price of an item (and express price)
6	orders/models.py	122	Coupon rules
7	orders/models.py	170	Order table and its safety rules
8	orders/pricing.py	65	Bill calculator (estimate)
9	orders/services.py	23	Allowed status steps
10	orders/services.py	92	Slot booking without overbooking (row lock)
11	orders/services.py	127	Change status (the ONLY door for status changes)
12	orders/services.py	170	Create an order (one transaction)
13	orders/migrations/0002_status_history_trigger.py	18	PostgreSQL TRIGGER: the automatic diary
14	tagging/models.py	14	Garment table: one row for every piece of clothing
15	tagging/services.py	55	Tag a garment (gives the next tag number)
16	tagging/services.py	111	Count check and mismatch alert
17	tagging/services.py	159	Final bill from the tagged garments
18	tagging/templatetags/qr.py	12	QR code for each tag
19	delivery/services.py	100	Delivery + Cash on Delivery (all in one go)
20	payments/invoice.py	46	PDF invoice
21	orders/notifications.py	47	Email notifications
22	dashboard/migrations/0001_daily_revenue_view.py	16	PostgreSQL VIEW: the revenue report
23	templates/orders/book.html	15	HTMX: page parts that update by themselves
24	core/management/commands/seed_demo.py	104	Demo data command
25	orders/tests.py	137	Automated tests (proof that it works)

Quick answer: "If the teacher asks ... open ..."

Teacher asks	Open
Where is the database connection?	washo/settings.py
Which address opens which page?	washo/urls.py
Show a table / constraints	orders/models.py (Order), accounts/models.py (User)
How is the bill calculated?	orders/pricing.py, tagging/services.py (finalise_bill)
How are slots protected?	orders/services.py (reserve_slot)
Where is the trigger?	orders/migrations/0002_status_history_trigger.py

Teacher asks	Open
Where is the view?	dashboard/migrations/0001_daily_revenue_view.py
What is unique in your project?	tagging/models.py (Garment), tagging/services.py (record_count)
How do you check roles?	accounts/permissions.py
How is payment recorded?	delivery/services.py (confirm_delivery), payments/models.py
How is the PDF made?	payments/invoice.py
How did you test it?	orders/tests.py, tagging/tests.py (run: manage.py test)

Part 2: What each file does, explained simply

Each block shows the real code from your project (with line numbers), then a simple explanation.

1. Settings: how WashO connects to the database

washo/settings.py · line 70

Teacher may ask: "Where is the database connection?" / "Where are passwords kept?"

```
70 DATABASES = {
71     "default": {
72         "ENGINE": "django.db.backends.postgresql",
73         "NAME": config("DB_NAME"),
74         "USER": config("DB_USER"),
75         "PASSWORD": config("DB_PASSWORD"),
76         "HOST": config("DB_HOST", default="localhost"),
77         "PORT": config("DB_PORT", default="5432"),
78         "CONN_MAX_AGE": 60, # reuse connections instead of reconnecting on every request
79     }
```

In simple words

Settings is like the instruction sheet stuck on the fridge. It tells WashO: "Our database is PostgreSQL, its name is washo, and the password is in the secret .env file." The real password is never written in the code. The code only says "go and read it from .env", the same way you'd keep your locker key in your pocket and not write the code on the locker.

2. URL map: which address opens which page

washo/urls.py · line 13

Teacher may ask: "How does Django know which page to show?"

```
13 urlpatterns = [
14     path("admin/", admin.site.urls),
15     path(
16         "login/",
17         auth_views.LoginView.as_view(authentication_form=PhoneLoginForm, redirect_authenticated-
18         name="login",
19     ),
20     path("logout/", auth_views.LogoutView.as_view(), name="logout"), # POST only (safer)
21     path("account/", include("accounts.urls")),
22     path("prices/", include("catalog.urls")),
23     path("stores/", include("stores.urls")),
24     path("orders/", include("orders.urls")),
25     path("staff/", include("tagging.urls")),
26     path("agent/", include("delivery.urls")),
27     path("payments/", include("payments.urls")),
28     path("dashboard/", include("dashboard.urls")),
29     path("", include("core.urls")),
```

In simple words

This is like the list of rooms in a school. If you go to /orders/ you reach the Orders room; /staff/ is the Staff room; /admin/ is the Principal's office. Django reads the address you typed, looks it up in this list and sends you to the right room (the right view).

3. Custom user: login with a mobile number

accounts/models.py · line 37

Teacher may ask: "Why a custom user model?" / "How is login done?"

```
37 class User(AbstractUser):
38     """Custom user: logs in with mobile number; email is optional.
39
40     WHY a custom model from day one: Django cannot easily switch the user model
41     after the first migration, and our users are identified by phone (as is
42     normal for Indian service apps), not by a username.
43     """
44
45     username = None # remove the username field inherited from AbstractUser
46     phone = models.CharField(
47         "mobile number",
48         max_length=10,
49         unique=True, # unique also creates a DB index -> fast login lookups
50         validators=[phone_validator],
51         help_text="10-digit mobile number, used to log in.",
52     )
53     email = models.EmailField("email address", blank=True)
54     # Store Staff and Delivery Agents work at one store; customers leave this empty.
55     store = models.ForeignKey("stores.Store", on_delete=models.PROTECT, null=True, blank=True,
56                             related_name="team", help_text="Only for Store Staff and Deli-
57
58     USERNAME_FIELD = "phone"
59     REQUIRED_FIELDS = [] # asked by `createsuperuser` besides phone + password
60
61     objects = UserManager()
62
63     class Meta:
64         ordering = ["first_name", "last_name"]
65         constraints = [
66             # Email is optional, but if given it must not belong to someone else.
```

In simple words

Normally a website asks for a username. In India people remember their phone number, so we made the phone number the login name. We also told the database two rules: a phone number must be 10 digits starting with 6, 7, 8 or 9, and two people can't share the same email. Even if someone tries to cheat and skip our form, the database itself says "No!".

4. Role check: who is allowed into which page

accounts/permissions.py · line 13

Teacher may ask: "How do you stop a customer opening the staff panel?"

```
13 def role_required(*roles):
14     """Decorator for function-based views: @role_required(Role.STAFF, Role.ADMIN)."""
15
16     def decorator(view_func):
17         @wraps(view_func)
18         def wrapper(request, *args, **kwargs):
19             if not request.user.is_authenticated:
20                 return redirect_to_login(request.get_full_path())
21             if not request.user.has_role(*roles):
22                 raise PermissionDenied
23             return view_func(request, *args, **kwargs)
24
25         return wrapper
26
27     return decorator
```

In simple words

Think of a security guard at a door with a list. Before letting you in, the guard asks two things: "Are you logged in?" (if not, go to the login page) and "Is your name on this door's list?" (if not, "Access denied", which is error 403). We put this guard on every page.

5. Price of an item (and express price)

catalog/models.py · line 109

Teacher may ask: "How is the express price calculated?"

```
109     def unit_price(self, express=False):
110         """Price for one unit. Rounded to paise with ROUND_HALF_UP (normal money rounding).~
111         price = self.price
112         if express and self.category.express_available:
113             price = price * (1 + self.category.express_surcharge_percent / Decimal("100"))
114         return price.quantize(TWO_PLACES, rounding=ROUND_HALF_UP)
```

In simple words

Each item has a price for each service, like a menu card: Shirt with Wash & Iron is Rs. 35. If you want it fast (Express), we add an extra percentage, like paying extra for a fast courier. The money is rounded to paise the normal way.

6. Coupon rules

orders/models.py · line 122

Teacher may ask: "How do coupons work?" / "How do you stop coupon misuse?"

```
122     def check_usable(self, user, order_amount, today):
123         """Raise CouponError (with the reason) if this coupon can't be applied."""
124         if not self.is_active:
125             raise CouponError("This coupon is no longer active.")
126         if today < self.valid_from:
127             raise CouponError(f"This coupon starts on {self.valid_from:%d %b %Y}.")
128         if today > self.valid_until:
129             raise CouponError("This coupon has expired.")
130         if self.usage_limit is not None and self.times_used() >= self.usage_limit:
131             raise CouponError("This coupon has been fully used.")
132         if user is not None and self.per_user_limit and \
133             self._used_orders().filter(customer=user).count() >= self.per_user_limit:
134             raise CouponError("You have already used this coupon.")
135         if order_amount < self.min_order_value:
136             short = money(self.min_order_value - order_amount)
137             raise CouponError(f"Add items worth Rs.{short} more to use this coupon (minimum~
138
139     def discount_for(self, amount):
140         """Discount in Rs. for an order amount; never more than the amount itself."""
141         if amount < self.min_order_value:
142             return Decimal("0.00")
143         if self.discount_type == self.DiscountType.FLAT:
144             discount = self.value
145         else:
146             discount = amount * self.value / Decimal("100")
147             if self.max_discount is not None:
148                 discount = min(discount, self.max_discount)
149         return money(min(discount, amount))
```

In simple words

A coupon is like a discount ticket at a fair. Before accepting it, we check: Is it switched on? Is today between its start and end date? Has it been used too many times overall? Did YOU already use it? Is your bill big enough? If all answers are OK, we give the discount, but never more than your bill.

7. Order table and its safety rules

orders/models.py · line 170

Teacher may ask: "Show me your main table" / "What constraints did you use?"

```
170 class Order(models.Model):
171     Status = OrderStatus # so code can say Order.Status.BOOKED
172
173     # NULL until the row exists (the number is built from the id). Unique ignores NULLs,
174     # so two orders being created at the same moment never clash.
175     code = models.CharField(max_length=12, unique=True, null=True, blank=True,
176                             help_text="Order number shown to customers.")
177     customer = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.PROTECT, relate~
178     address = models.ForeignKey("accounts.Address", on_delete=models.PROTECT, related_name=~
179     # WHY a text copy too: the order (and its invoice) must keep the address as it was
180     # on the booking day, even if the customer edits the saved address later.
181     address_snapshot = models.TextField()
```

also orders/models.py · line 224

```
224         condition=Q(total=F("subtotal") + F("express_charge") - F("discount")),
225         name="order_total_adds_up",
226     ),
```

In simple words

The Order table is the main notebook page for each order: who booked it, from where, which slot, which status, and the money. At the bottom we wrote a maths rule inside the database: total must equal subtotal + express charge - discount. If any program ever saves wrong maths, the database refuses, like a teacher who won't accept a wrong answer.

8. Bill calculator (estimate)

orders/pricing.py · line 65

Teacher may ask: "How is the total calculated?"

```
65 def build_quote(selected, is_express=False, coupon_code="", user=None, today=None):
66     """Work out the bill for a list of (ServicePrice, quantity) pairs.
67
68     The coupon is checked against the gross amount (after express, before discount).
69     An invalid coupon does NOT stop the quote; the reason is put in `coupon_error`.
70     """
71     today = today or timezone.localdate()
72     quote = Quote(is_express=is_express)
73     for service_price, qty in selected:
74         if qty <= 0:
75             continue
76         line = QuoteLine(service_price, qty)
77         quote.lines.append(line)
78         base = money(qty * line.unit_price)
79         extra = money(qty * line.express_surcharge(is_express))
80         line.amount = base + extra
81         quote.subtotal += base
82         quote.express_charge += extra
83
84     coupon_code = (coupon_code or "").strip().upper()
85     if coupon_code:
86         coupon = Coupon.objects.filter(code=coupon_code).first()
87         if coupon is None:
88             quote.coupon_error = "This coupon code doesn't exist."
89         else:
90             try:
91                 coupon.check_usable(user, quote.gross, today)
92                 quote.coupon = coupon
93                 quote.discount = coupon.discount_for(quote.gross)
94             except CouponError as e:
95                 quote.coupon_error = str(e)
96     return quote
```

In simple words

This is like a shopkeeper's calculator. For each item it does quantity x price. Then it adds the express extra (only for services that offer express), then takes away the coupon discount. It doesn't save anything; it only calculates, which is why it's easy to test.

9. Allowed status steps

orders/services.py · line 23

Teacher may ask: "How do you control the order flow?"

```
23 ALLOWED_TRANSITIONS = {
24     S.BOOKED: {S.PICKUP_ASSIGNED, S.CANCELLED},
25     S.PICKUP_ASSIGNED: {S.PICKED_UP, S.CANCELLED},
26     S.PICKED_UP: {S.AT_STORE},
27     S.AT_STORE: {S.TAGGED},
28     S.TAGGED: {S.IN_CLEANING},
29     S.IN_CLEANING: {S.READY},
30     S.READY: {S.OUT_FOR_DELIVERY},
31     S.OUT_FOR_DELIVERY: {S.DELIVERED},
32     S.DELIVERED: set(),
33     S.CANCELLED: set(),
34 }
35 # Cancelling is only possible before the clothes leave the customer's home.
36 CANCELLABLE_STATUSES = {S.BOOKED, S.PICKUP_ASSIGNED}
```

In simple words

An order moves like a board game piece: Booked -> Pickup Assigned -> Picked Up -> At Store -> Tagged -> In Cleaning -> Ready -> Out for Delivery -> Delivered. This list says which square you may move to next. You can't jump from Booked straight to Ready, and once Delivered the game is over. Cancel is only allowed before the clothes leave your home.

10. Slot booking without overbooking (row lock)

orders/services.py · line 92

Teacher may ask: "What if two people book the last slot at the same time?"

```
92 def reserve_slot(date, time_slot, area):
93     """Take one place in the slot or raise BookingError if it is full.
94
95     Must run inside a transaction. select_for_update() locks the counter row,
96     so concurrent bookings for the same slot queue up one after another.
97     """
98     daily, _ = DailySlot.objects.select_for_update().get_or_create(date=date, time_slot=time-
99     if daily.booked_count >= time_slot.capacity:
100         raise BookingError("Sorry, this slot just got full. Please choose another time.")
101     daily.booked_count = F("booked_count") + 1 # done by the DB: no lost updates
102     daily.save(update_fields=["booked_count"])
103     daily.refresh_from_db()
104     return daily
```

In simple words

Imagine a slide with only 5 turns left and two kids running for the last turn. The database puts a "hold" (a lock) on the counter. The first kid takes the turn, then the second kid is let in, sees "0 left" and is told "Sorry, full". That's SELECT ... FOR UPDATE: only one person can change the counter at a time.

11. Change status (the ONLY door for status changes)

orders/services.py · line 127

Teacher may ask: "Where do you change an order's status?"

```
127 def change_status(order, new_status, by, note=""):
128     """The ONLY way order status changes. Validates the move; the DB trigger logs it."""
129     order = Order.objects.select_for_update().get(pk=order.pk) # stop two people changing ~
130     new_status = S(new_status)
131     if new_status == S.CANCELLED and order.status not in CANCELLABLE_STATUSES:
132         raise InvalidTransition("This order can't be cancelled because the clothes have alr~
133     if new_status not in ALLOWED_TRANSITIONS[S(order.status)]:
134         raise InvalidTransition(f"Can't change status from {order.get_status_display()} to ~
135     # Someone must be responsible for the trip before it starts.
136     if new_status == S.PICKUP_ASSIGNED and not order.pickup_agent_id:
137         raise InvalidTransition("Assign a pickup agent first (Staff panel -> order -> Assig~
138     if new_status == S.OUT_FOR_DELIVERY and not order.delivery_agent_id:
139         raise InvalidTransition("Assign a delivery agent first (Staff panel -> order -> Ass~
140
141     _set_audit_context(by, note)
142     order.status = new_status
143     order.save(update_fields=["status", "updated_at"])
144
145     if new_status == S.CANCELLED:
146         release_slot(order.pickup_slot) # give the pickup place back to other customers
147     # Email the customer only after the change is safely saved (and never twice if it rolls~
148     transaction.on_commit(lambda pk=order.pk, st=new_status: send_order_email(pk, st))
149     return order
```

In simple words

There is only ONE door to change an order's status, and this is it. At the door it checks: is this step allowed? Is an agent assigned before a trip? Then it whispers to the database "Ravi is doing this" (set_config), changes the status, and after saving, sends the customer an email.

12. Create an order (one transaction)

orders/services.py · line 170

Teacher may ask: "What happens when the customer clicks Confirm pickup?"

```
170 def create_order(*, customer, address, pickup_date, time_slot, selected_items, is_express=F~
171         coupon_code="", note="", now=None):
172     """Create an order: reserve the slot, check the coupon, copy prices, save everything.
173
174     Everything is in ONE transaction: if any step fails (slot full, coupon used
175     up), nothing is saved and the slot place is not taken.
176     """
177     if address.user_id != customer.pk or not address.is_active:
178         raise BookingError("Please choose one of your saved addresses.")
179     if not any(qty > 0 for _, qty in selected_items):
180         raise BookingError("Please add at least one item.")
181
182     check_slot_bookable(pickup_date, time_slot, now)
183     daily_slot = reserve_slot(pickup_date, time_slot, address.area)
184
185     coupon_code = (coupon_code or "").strip().upper()
186     if coupon_code:
187         # Lock the coupon so its usage limit can't be exceeded by simultaneous bookings.
188         Coupon.objects.select_for_update().filter(code=coupon_code).first()
189     quote = build_quote(selected_items, is_express, coupon_code, customer, timezone.localda~
190     if quote.coupon_error:
191         raise BookingError(quote.coupon_error)
192
193     _set_audit_context(customer, "Order placed")
194     order = Order.objects.create(
195         customer=customer, address=address, address_snapshot=address.one_line(),
196         store=address.area.store, pickup_slot=daily_slot, is_express=is_express,
197         coupon=quote.coupon, customer_note=note,
198         estimated_total=quote.total, subtotal=quote.subtotal, express_charge=quote.express~
199         discount=quote.discount, total=quote.total,
200     )
201     OrderItem.objects.bulk_create([
202         OrderItem(order=order, category=line.category, item=line.item, quantity=line.quantit~
203             unit_price=line.unit_price, express_surcharge=line.express_surcharge(is_e~
204         for line in quote.lines
205     ])
206     order.code = f"W0-{order.pk:06d}"
207     order.save(update_fields=["code"])
208     transaction.on_commit(lambda pk=order.pk: send_order_email(pk, S.BOOKED))
209     return order
```

In simple words

Booking is done as ONE package, like a sealed envelope. Inside: take a slot place, check the coupon, copy today's prices, save the order and its items. If anything fails (for example the coupon is not valid), the whole envelope is torn up and nothing is saved, not even the slot place.

13. PostgreSQL TRIGGER: the automatic diary

orders/migrations/0002_status_history_trigger.py · line 18

Teacher may ask: "Show me a database feature you used" / "What is a trigger?"

```
18 CREATE OR REPLACE FUNCTION washo_log_order_status() RETURNS trigger AS $$
19 DECLARE
20     -- current_setting(name, true) returns NULL instead of an error when the setting is mis-
21     v_user_id bigint := NULLIF(current_setting('washo.changed_by', true), '')::bigint;
22     v_note     text  := COALESCE(current_setting('washo.status_note', true), '');
23     v_from     text  := '';
24 BEGIN
25     IF TG_OP = 'UPDATE' THEN
26         v_from := OLD.status;
27     END IF;
28
29     INSERT INTO orders_orderstatushistory (order_id, from_status, to_status, changed_by_id, ~
30     VALUES (NEW.id, v_from, NEW.status, v_user_id, LEFT(v_note, 300), now());
31
32     RETURN NEW;
33 END;
34 $$ LANGUAGE plpgsql;
35
36 -- Fires once for every new order (history starts with "Booked").
37 CREATE TRIGGER trg_order_status_on_insert
38     AFTER INSERT ON orders_order
39     FOR EACH ROW
40     EXECUTE FUNCTION washo_log_order_status();
41
42 -- Fires only when the status column really changes (not on other edits).
43 CREATE TRIGGER trg_order_status_on_update
44     AFTER UPDATE OF status ON orders_order
45     FOR EACH ROW
46     WHEN (OLD.status IS DISTINCT FROM NEW.status)
47     EXECUTE FUNCTION washo_log_order_status();
48 """"
```

In simple words

A trigger is a little robot that lives INSIDE the database. Whenever an order is created or its status changes, the robot wakes up by itself and writes a line in the history diary: old status, new status, who and when. Nobody can forget to write the diary, not even someone typing SQL directly, because the robot is always watching.

14. Garment table: one row for every piece of clothing

tagging/models.py · line 14

Teacher may ask: "What is special about your project?"

```
14 class Garment(models.Model):
15     """ONE physical garment, tagged at the store with its own code + QR.
16
17     WHY one row per garment (not just a count): we can show the customer exactly
18     which pieces we received, note stains/damage on the right piece, and prove the
19     count at every step. This is our answer to the "missing clothes" complaint.
20     """
21
22     order = models.ForeignKey("orders.Order", on_delete=models.PROTECT, related_name="garment-
23     seq = models.PositiveSmallIntegerField(help_text="1, 2, 3 ... within the order.")
24     tag_code = models.CharField(max_length=20, unique=True, help_text="Printed on the tag, ~
25     category = models.ForeignKey("catalog.ServiceCategory", on_delete=models.PROTECT)
26     item = models.ForeignKey("catalog.Item", on_delete=models.PROTECT)
27     # Price copied at tagging, so the final bill never changes afterwards.
28     unit_price = models.DecimalField(max_digits=8, decimal_places=2)
29     express_surcharge = models.DecimalField(max_digits=8, decimal_places=2, default=0)
30     description = models.CharField(max_length=120, blank=True, help_text="Colour / brand, e-
31
32     has_stain = models.BooleanField(default=False)
33     has_damage = models.BooleanField(default=False)
34     condition_note = models.CharField(max_length=300, blank=True,
35                                     help_text="Where and what, e.g. 'Oil stain on left sl-
36     photo = models.ImageField(upload_to="garments/%Y/%m/", blank=True, validators=[validate~
```

In simple words

Most laundries count a bag. We give every shirt its own name tag, like each student having a roll number. The row remembers which order it belongs to, what it is, its colour, and any stain or damage, with an optional photo. A rule says: if you tick "stain", you MUST write what the stain is.

15. Tag a garment (gives the next tag number)

tagging/services.py · line 55

Teacher may ask: "How is the tag code made?"

```
55 def add_garment(order, service_price, by, description="", has_stain=False, has_damage=False,
56                 condition_note="", photo=None):
57     order = _locked(order) # lock: two staff tagging the same order can't get the same num-
58     if order.status != S.AT_STORE:
59         raise TaggingError("Garments can only be tagged while the order is At Store.")
60     if (has_stain or has_damage) and not condition_note.strip():
61         raise TaggingError("Please describe the stain or damage.")
62     seq = (order.garments.aggregate(m=Max("seq"))["m"] or 0) + 1
63     if seq > MAX_GARMENTS_PER_ORDER:
64         raise TaggingError(f"An order can have at most {MAX_GARMENTS_PER_ORDER} garments.")
65     unit_price, surcharge = price_for(order, service_price)
66     return Garment.objects.create(
67         order=order, seq=seq, tag_code=f"{order.code}-{seq:02d}",
68         category=service_price.category, item=service_price.item,
69         unit_price=unit_price, express_surcharge=surcharge, description=description.strip(),
70         has_stain=has_stain, has_damage=has_damage, condition_note=condition_note.strip(),
71         photo=photo or "", tagged_by=by,
72     )
```

In simple words

When staff tag a piece, the code locks the order (so two staff don't get the same number), finds the next number (01, 02, 03 ...), picks the price the customer was promised, and saves the garment as <order code>-<number>.

16. Count check and mismatch alert

tagging/services.py · line 111

Teacher may ask: "How does the missing-clothes alert work?"

```
111 def record_count(order, checkpoint, count, by, note=""):
112     """Save the count for a checkpoint and raise an alert if it differs from the previous o-
113
114     Returns (CountCheck, MismatchAlert or None).
115     """
116     checkpoint = CP(checkpoint)
117     if count < 0:
118         raise TaggingError("Count can't be negative.")
119     check, _ = CountCheck.objects.update_or_create(
120         order=order, checkpoint=checkpoint,
121         defaults={"count": count, "recorded_by": by, "note": note},
122     )
123
124     alert = None
125     prev_cp = PREVIOUS_CHECKPOINT.get(checkpoint)
126     prev = CountCheck.objects.filter(order=order, checkpoint=prev_cp).first() if prev_cp el-
127     if prev and prev.count != count:
128         # Don't raise the same alert twice if the same wrong count is entered again.
129         alert = MismatchAlert.objects.filter(order=order, checkpoint=checkpoint, status=Mis-
130             expected_count=prev.count, actual_count=count)~
131         if alert is None:
132             alert = MismatchAlert.objects.create(order=order, checkpoint=checkpoint,
133                 expected_count=prev.count, actual_count=co-
134             transaction.on_commit(lambda a=alert: _notify_admin(a)) # email only if the sa-
135     return check, alert
```

In simple words

We count the clothes three times, like counting students when the bus leaves the school, when it reaches the zoo and when it comes back. Each count is compared with the one before it. If the numbers are different, an alarm (MismatchAlert) is created and the admin gets an email.

17. Final bill from the tagged garments

tagging/services.py · line 159

Teacher may ask: "Is the customer charged on the estimate?"

```
159 def finalise_bill(order):
160     """Replace the booking estimate with the bill for the garments actually tagged.
161
162     The coupon (if any) is applied again on the new amount; if the order is now
163     below the coupon's minimum value the discount becomes 0.
164     """
165     sums = order.garments.aggregate(sub=Sum("unit_price"), extra=Sum("express_surcharge"))
166     order.subtotal = money(sums["sub"] or 0)
167     order.express_charge = money(sums["extra"] or 0)
168     gross = order.subtotal + order.express_charge
169     order.discount = order.coupon.discount_for(gross) if order.coupon else Decimal("0.00")
170     order.total = gross - order.discount
171     order.bill_finalised = True
172     order.save(update_fields=["subtotal", "express_charge", "discount", "total", "bill_fina-
```

In simple words

At booking the customer only guesses ("about 10 shirts"). At the store we know the truth. So the final bill adds up the real garments. The coupon is checked again: if the bill became too small for the coupon, the discount goes away.

18. QR code for each tag

tagging/templatetags/qr.py · line 12

Teacher may ask: "How are QR codes generated?"

```
12 def qr_svg(data):
13     """QR code as an inline <svg> string (vector: sharp on screen and in print, no image fi-
14     img = qrcode.make(data, image_factory=qrcode.image.svg.SvgPathImage, box_size=10, borde-
15     buf = BytesIO()
16     img.save(buf)
17     svg = buf.getvalue().decode()
18     svg = svg[svg.index("<svg"):] # drop the <?xml ...?> line so it can sit inside HTML
19     return svg.replace("<svg ", '<svg class="qr" role="img" ', 1)
```

In simple words

The qrcode library turns text into a square barcode picture (drawn as SVG, which stays sharp when printed). Our QR contains the web address of that garment's page, so scanning it with a phone opens the garment's details.

19. Delivery + Cash on Delivery (all in one go)

delivery/services.py · line 100

Teacher may ask: "How is payment recorded?"

```
100 def confirm_delivery(order, agent, count, confirm_mismatch=False, note="", payment_method=N~
101     """At the customer's door: count the garments handed over and collect the bill (Cash on-
102
103     A different count needs confirmation and raises a mismatch alert for admin. Count, paym-
104     'Delivered' are saved in ONE transaction - all of them, or none.
105     """
106     order = _locked(order)
107     _check_own_job(order, agent, "delivery_agent")
108     if order.status != S.OUT_FOR_DELIVERY:
109         raise DeliveryError("This order is not out for delivery.")
110     expected = expected_count(order, CP.DELIVERY)
111     if expected is not None and count != expected and not confirm_mismatch:
112         raise DeliveryError(
113             f"The store tagged {expected} garments but you counted {count}. Count again wit-
114             "if it is really different, tick the confirmation box (admin will be alerted)."
115         )
116     due = amount_due(order)
117     if due > 0 and not payment_method:
118         raise DeliveryError(f"Collect Rs.{due} from the customer (cash or UPI) and tick 'Pa-
119         record_count(order, CP.DELIVERY, count, agent, note)
120     if due > 0:
121         try:
122             record_collection(order, agent, payment_method, reference)
123         except PaymentError as e:
124             raise DeliveryError(str(e))
125     return change_status(order, S.DELIVERED, agent, note=f"Delivered {count} garments" + (f~
```

In simple words

At the customer's door the agent counts the clothes and collects the money. The code saves three things together: the delivery count, the payment (cash or UPI) and the status "Delivered". If the money isn't collected, the order can't become Delivered. Either everything is saved or nothing.

20. PDF invoice

payments/invoice.py · line 46

Teacher may ask: "How is the invoice made?"

```
46 def build_invoice_pdf(order):
47     """Return the invoice as PDF bytes."""
48     buf = BytesIO()
49     doc = SimpleDocTemplate(buf, pagesize=A4, leftMargin=18 * mm, rightMargin=18 * mm,
50                             topMargin=16 * mm, bottomMargin=16 * mm,
51                             title=f"WashO invoice {invoice_number(order)}", author="WashO")
52     ss = getSampleStyleSheet()
53     normal = ParagraphStyle("n", parent=ss["Normal"], fontSize=9.5, leading=13)
54     small = ParagraphStyle("s", parent=normal, fontSize=8, textColor=colors.grey)
55     right = ParagraphStyle("r", parent=normal, alignment=TA_RIGHT)
56     h_brand = ParagraphStyle("b", parent=ss["Title"], fontSize=22, leading=26, textColor=BR-
57                             spaceBefore=0, spaceAfter=2)
58
59     store = order.store
```

In simple words

ReportLab is like a robot printer that draws a page using Python: the shop's name at the top, the customer, a table of items, the totals and PAID or PAYMENT DUE. Everything is drawn in memory and given to the browser as a PDF file.

21. Email notifications

orders/notifications.py · line 47

Teacher may ask: "How are customers informed?"

```
47 def send_order_email(order_id, status):
48     """Build and send the email for the status the order just moved to."""
49     order = Order.objects.select_related("customer", "store", "pickup_slot__time_slot",
50                                         "pickup_agent", "delivery_agent").get(pk=order_id)
51     email = order.customer.email
52     if not email:
53         return False # email is optional for customers
54     subject_text, line = MESSAGES[S(status)]
55     ctx = {
56         "order": order,
57         "customer": order.customer,
58         "headline": subject_text,
59         "message": line.format(**_context(order)),
60         "order_url": settings.SITE_URL.rstrip("/") + reverse("orders:detail", args=[order.c~
61         "site_url": settings.SITE_URL,
62     }
63     msg = EmailMultiAlternatives(
64         subject=f"WashO {order.code}: {subject_text}",
65         body=render_to_string("emails/order_status.txt", ctx),
66         from_email=settings.DEFAULT_FROM_EMAIL,
67         to=[email],
68     )
69     msg.attach_alternative(render_to_string("emails/order_status.html", ctx), "text/html")
70     if S(status) == S.DELIVERED and order.bill_finalised:
```

In simple words

After every status change we send a short letter (email) to the customer: "Your clothes are picked up", "Your bill is ready" and so on. It is sent only AFTER the change is safely saved, and if the post office (mail server) is broken, the order is still fine.

22. PostgreSQL VIEW: the revenue report

dashboard/migrations/0001_daily_revenue_view.py · line 16

Teacher may ask: "Show me your second database feature" / "What is a view?"

```
16 CREATE OR REPLACE VIEW dashboard_daily_revenue AS
17 SELECT
18     row_number() OVER (ORDER BY t.day, t.store_id) AS id,
19     t.day,
20     t.store_id,
21     count(*)           AS orders,
22     sum(t.amount)      AS revenue
23 FROM (
24     SELECT (p.collected_at AT TIME ZONE 'Asia/Kolkata')::date AS day,
25            o.store_id,
26            p.amount
27     FROM payments_payment AS p
28     JOIN orders_order     AS o ON o.id = p.order_id
29 ) AS t
30 GROUP BY t.day, t.store_id;
31 """
32
33 DROP_VIEW = "DROP VIEW IF EXISTS dashboard_daily_revenue;"
```

In simple words

A view is a saved question we keep asking the database: "How much money did each store collect each day?" It looks like a table, but it has no data of its own. Every time you look at it, it calculates the answer fresh, like a calculator that always shows today's total.

23. HTMX: page parts that update by themselves

templates/orders/book.html · line 15

Teacher may ask: "How does the price update without reloading?"

```
15         hx-post="{% url 'orders:book_estimate' %}" hx-target="#estimate"
16         hx-trigger="change, input delay:500ms">
17         {% csrf_token %}
```

also templates/orders/_order_status.html · line 7

```
7         {% if order.is_open %}hx-get="{% url 'orders:status_partial' order.code %}" hx-trigger~
```

In simple words

HTMX is like a waiter who brings only the dish you changed, not the whole meal again. When you type an item count, the browser quietly asks the server for a new estimate box and swaps just that box. The order page asks every 15 seconds "any news?" and updates the status by itself.

24. Demo data command

core/management/commands/seed_demo.py · line 104

Teacher may ask: "Where does this data come from?"

```
104 class Command(BaseCommand):
105     help = "Load realistic Pune demo data (staff, agents, 20 customers, 60 orders in every ~
106
107     def add_arguments(self, parser):
108         parser.add_argument("--reset", action="store_true", help="Delete existing demo data~
109
110     def handle(self, *args, **opts):
111         if opts["reset"]:
112             self.reset()
113         elif demo_users().exists():
114             self.stdout.write(self.style.WARNING("Demo data is already loaded - nothing to ~
115                                     "(Use --reset to delete and rebuild it.)"))
116         return
117         self.rng = random.Random(2026) # fixed seed: the same demo data on every computer
118         self.now = timezone.now()
119         self.today = timezone.localtime()
120         # Emails would be sent for ~300 status changes; switch them off while seeding.
121         with override_settings(EMAIL_BACKEND="django.core.mail.backends.dummy.EmailBackend"~
122             self.make_people()
123             made = self.make_orders()
```

In simple words

seed_demo is like a puppet show: it pretends to be 20 customers, staff and agents, and makes 60 orders by pressing the same buttons a real person would. So all the rules, counts and history are real. Then it moves the dates back, so the charts show six weeks of business.

25. Automated tests (proof that it works)

orders/tests.py · line 137

Teacher may ask: "How did you test it?"

```
137 class SlotCapacityTests(TestCase):
138     def setUp(self):
139         self.slot = TimeSlot.objects.get(start_time=datetime.time(10)) # capacity 5
140         self.area = ServiceArea.objects.get(name="Kothrud")
141
142     def test_slot_cannot_be_overbooked(self):
143         with transaction.atomic():
144             for _ in range(self.slot.capacity):
145                 reserve_slot(TOMORROW, self.slot, self.area)
146         with self.assertRaisesMessage(BookingError, "just got full"):
147             with transaction.atomic():
148                 reserve_slot(TOMORROW, self.slot, self.area)
149         self.assertEqual(DailySlot.objects.get(date=TOMORROW, time_slot=self.slot, area=self~
```

In simple words

Tests are like a robot student who answers the same exam questions every time we change something. We have 138 questions, for example "Can a full slot take one more booking?" The answer must be "No". If any answer is wrong, we know straight away.

Part 3: Examples

Real numbers from WashO. Say these out loud in the viva: examples make answers easy to believe.

1. Settings: how WashO connects to the database

The line `NAME: config("DB_NAME")` means: open `.env`, find `DB_NAME=washo`, and use "washo". On the second laptop the same code works with a different password, because only `.env` changes.

2. URL map: which address opens which page

Typing `127.0.0.1:8000/prices/` matches `path("prices/", ...)` and opens the price list page.

3. Custom user: login with a mobile number

Typing `+91 98765 43210` is cleaned to `9876543210` and you can log in. Trying to save phone `1234567890` fails, because it starts with 1 and the `CHECK` rule blocks it.

4. Role check: who is allowed into which page

`@role_required(Role.STAFF)` on the staff panel: Store Staff get in, but a customer gets the 403 "Access denied" page.

5. Price of an item (and express price)

Silk saree Dry Clean = Rs. 299. Express is +50%, so $299 \times 1.5 = \text{Rs. } 448.50$.

6. Coupon rules

`WELCOME50` = Rs. 50 off if the bill is at least Rs. 299. A Rs. 250 bill shows "Add items worth Rs. 49.00 more". `FRESH20` = 20% off, max Rs. 150: a Rs. 2,000 bill gets Rs. 150 off, not Rs. 400.

7. Order table and its safety rules

$\text{Subtotal } 439 + \text{express } 219.50 - \text{discount } 131.70 = \text{total } 526.80$, so it is accepted. Trying to save total = 1 is refused with an `IntegrityError`.

8. Bill calculator (estimate)

5 shirts x Rs. 35 = 175, plus 1 silk saree Rs. 299 = Rs. 474. With express: +Rs. 237. With `FRESH20`: -20% (max Rs. 150).

9. Allowed status steps

Booked -> Ready is rejected with "Can't change status from Booked to Ready".

10. Slot booking without overbooking (row lock)

The 10-12 slot in Kothrud has capacity 5. Bookings 1-5 succeed. Booking 6 gets "Sorry, this slot just got full". Cancelling one booking gives the place back.

11. Change status (the ONLY door for status changes)

Staff clicks "Start cleaning": Tagged -> In Cleaning is allowed, so it's saved, the history row is written and an email is sent.

12. Create an order (one transaction)

A Rs. 35 order with `WELCOME50` (needs Rs. 299) fails, and the slot counter stays exactly the same. That's a "transaction rollback".

13. PostgreSQL TRIGGER: the automatic diary

Agent confirms pickup: the diary gets "pickup_assigned -> picked_up, by Ravi, 10:32 AM". In the tests we changed the status with plain SQL and the robot still wrote it.

14. Garment table: one row for every piece of clothing

Tag WO-000012-03 = Silk saree, Dry Clean, "Turmeric stain near hem", with a photo.

15. Tag a garment (gives the next tag number)

Order WO-000012 already has 2 garments, so the next one becomes WO-000012-03.

16. Count check and mismatch alert

The agent counted 5 at pickup. The store tags only 4. An alert is made: "expected 5, counted 4, -1", and the admin is emailed.

17. Final bill from the tagged garments

Booked 3 shirts (estimate Rs. 105); the store found 4 shirts + 1 saree, so the final bill is $4 \times 35 + 299 =$ Rs. 439.

18. QR code for each tag

The tag WO-000012-03 holds the address `.../staff/tag/WO-000012-03/` inside its QR.

19. Delivery + Cash on Delivery (all in one go)

Bill Rs. 369. Without ticking "Payment collected" you see "Collect Rs. 369.00 ...". With the tick: a payment of Rs. 369 cash is saved and the order is Delivered.

20. PDF invoice

INV-WO-000006: Saree Rs. 299 + 4 shirts Rs. 140 = Rs. 439; express +219.50; FRESH20 -131.70; total Rs. 526.80, PAID by cash.

21. Email notifications

Out for Delivery email: "Ravi is on the way. Please keep Rs. 526.80 ready (cash or UPI)."

22. PostgreSQL VIEW: the revenue report

`SELECT * FROM dashboard_daily_revenue;` gives one row per store per day. A real row from your laptop: 2026-10-01 | WashO Baner | 1 order | Rs. 1,058.00 (dates move if the demo data is rebuilt).

23. HTMX: page parts that update by themselves

Type 5 next to Shirt and the estimate changes to Rs. 175 without the page blinking.

24. Demo data command

`venv\Scripts\python manage.py seed_demo --reset` rebuilds all the demo data in about 20 seconds.

25. Automated tests (proof that it works)

`venv\Scripts\python manage.py test -> "Ran 138 tests ... OK".`

One full story: an order from start to end

1. **Asha books** 4 shirts (Wash & Iron) and 1 silk saree (Dry Clean), Express, coupon FRESH20. Estimate: $140 + 299 = 439$; express +219.50; coupon -131.70 = **Rs. 526.80**. (orders/pricing.py)
2. The 8-10 AM Kothrud slot had 5 places; now 4 are left. (orders/services.py `reserve_slot`)
3. Staff assign agent **Ravi**: status Pickup Assigned. The trigger writes it in the diary.
4. Ravi counts **5 garments** with Asha: status Picked Up. (delivery/services.py `confirm_pickup`)
5. The store tags 5 pieces, WO-000006-01 ... -05; one saree has a stain note + photo. (tagging/services.py)
6. Store count 5 = pickup count 5, so **no alert**. Final bill Rs. 526.80. Status Tagged.

7. Cleaning, then Ready. Ravi is assigned for delivery and starts: Out for Delivery. Asha gets an email "keep Rs. 526.80 ready".
8. At the door Ravi counts 5, collects **Rs. 526.80 cash** and ticks Paid: status Delivered, payment saved, and the invoice PDF is emailed. (delivery/services.py confirm_delivery)
9. The admin's dashboard shows the Rs. 526.80 in today's revenue through the PostgreSQL view.
If the store had found only 4 pieces: an alert "expected 5, counted 4" goes to the admin, who checks and writes how it was resolved.